Aegis Entropy

AI-Based Intelligent System for Network Security Monitoring & Automated Response

PORT SCANNING DETECTION • MTD • HONEYPOT

Module: Artificial Intelligence | Status: Academic Project | Severity: MEDIUM-HIGH

Project Team

El Kartouti Anas
Hammoubel El Yazid
Lekhbioui Adil
Moulay Anas
Nafia Khadija

Academic Year 2025 – 2026

Table of Contents

PART I

Needs Expression

> **Document origin:** This part is derived from the initial Needs Expression document (*Aegis-Morph* phase) which establishes the academic, methodological, and contextual foundations of the Aegis Entropy project.

1. Project Contextualisation

1.1 General Context

The digital transformation of organisations across all sectors — public administration, healthcare, finance, industry, and education — has led to an unprecedented expansion of networked infrastructures. Whether deployed as Local Area Networks (LAN), Wide Area Networks (WAN), Metropolitan Area Networks (MAN), wireless networks, cloud environments, or hybrid architectures, these infrastructures now serve as the backbone of critical operations, enabling resource sharing, real-time communication, and distributed service delivery.

This ubiquity, however, comes at a significant cost: the larger and more complex a network, the broader its attack surface. Network security has consequently become one of the most pressing challenges in modern information systems management. Malicious actors continuously evolve their strategies, exploiting vulnerabilities at every layer of the network stack — from the physical and data-link layers up through the application layer.

In response, the field of Network Intrusion Detection and Prevention Systems (IDS/IPS) has emerged as a critical discipline. Traditional approaches rely on predefined signatures and static rule sets. While effective against known threats, these systems fail to anticipate novel, adaptive, or previously unseen attack patterns. The limitations of such approaches have created a clear and growing demand for intelligent, self-learning security systems capable of analysing network behaviour dynamically and responding autonomously.

2. The Role of Artificial Intelligence in Network Security

Artificial Intelligence, and more specifically machine learning, offers a fundamentally different approach to intrusion detection. Rather than relying on manually curated rules, AI-based systems learn statistical models of normal network behaviour from labelled traffic datasets. Any significant deviation from the learned baseline is flagged as a potential threat — enabling the detection of both known and previously uncharacterised attack vectors.

This paradigm shift brings several decisive advantages over classical IDS/IPS solutions: adaptability to evolving threats, the ability to process and classify high-volume traffic in real time, reduced dependency on manual signature updates, and the capacity to support autonomous decision-making — including automated threat containment without human intervention.

Classical IDS/IPS	AI-Based IDS/IPS
Relies on predefined signatures	Learns behavioural patterns from data
Fails against unknown/novel attacks	Detects anomalies beyond known signatures
Requires constant manual updates	Self-adapts through model retraining
No autonomous response capability	Supports automated threat neutralisation
High false positive rate at scale	Optimised precision/recall via ML tuning
Static and network-topology-specific	Generalisable across diverse network types

Table 1.1 — Classical IDS/IPS vs. AI-Based IDS/IPS

3. Project Framework & Methodology

This project is carried out within the Artificial Intelligence academic module by a team of five students. Its objective is to design, implement, test, and validate an AI-powered IDS/IPS system, deployed over a simulated network topology spanning multiple nodes and traffic types. The system must be capable of monitoring network traffic in real time, detecting suspicious behavioural patterns, profiling threat actors, triggering alerts, and enforcing automated countermeasures.

The project is structured around the **PDCA (Plan–Do–Check–Act)** continuous improvement methodology, which ensures that each development phase is rigorously planned, executed, evaluated, and refined before progression to the next stage.

Phase	Description
Plan	Define project objectives, identify data sources, design the network topology, select AI models and evaluation criteria, and structure the development roadmap.
Do	Implement the full system pipeline: data preprocessing, model training, real-time traffic capture, threat detection engine, dashboard, and automated response module.
Check	Evaluate the system against simulated threat scenarios. Measure performance against predefined metrics (accuracy, latency, false positive rate). Document observations and gaps.
Act	Validate findings through cross-validation and end-to-end testing. Identify limitations, propose improvements, and consolidate results into the final academic deliverable.

Table 1.2 — PDCA Methodology Phases

4. Design Methodology: CPS (Context – Problematic – Solution)

Beyond the overarching PDCA framework, the project adopts the **CPS (Context–Problematic–Solution)** scientific design methodology to ensure that every technical decision is grounded in observed, field-validated needs rather than assumptions.

Phase	Description

Context	Establishes the environment in which the project operates: the nature of modern network infrastructures, the evolving threat landscape, and the limitations of existing security tools. This Needs Expression document constitutes the Context phase.
Problematic	Derived from structured interviews conducted with network and infrastructure professionals. An interview guide will be developed, administered, and transcribed into formal interview reports. The technical problematic will be formalised exclusively from the insights gathered during this phase.
Solution	A targeted technical solution is proposed based on the findings of the Problematic phase. It specifies the chosen approach, tools, datasets, model architecture, evaluation metrics, and the integration of advanced defence mechanisms into the IDS/IPS system.

Table 1.3 — CPS Methodology Phases

5. Network Scope

The system is designed to be applicable across a broad spectrum of network architectures and deployment contexts. While the academic prototype will be validated in a controlled simulation environment, the principles, models, and mechanisms developed are intended to generalise to real-world infrastructure of any type and scale.

Network Type	Description	Relevance
LAN	Local Area Network — controlled, high-speed, limited geographic scope	Primary simulation environment for prototype validation
WAN	Wide Area Network — geographically distributed, multi-site connectivity	Relevant for enterprise and inter-organisational threat detection
MAN	Metropolitan Area Network — city-scale infrastructure	Applicable to smart city, utility, and campus-wide deployments
Wireless / Wi-Fi	IEEE 802.11 networks — mobile, access-point-based	Relevant to BYOD environments and open-access networks
Cloud / Virtual	Software-defined and cloud-hosted network infrastructure	Growing target surface for network-level intrusions
Hybrid	Combination of on-premise and cloud, wired and wireless	Reflects most real-world organisational deployments

Table 1.4 — Applicable Network Types

6. Advanced Security Concepts Integrated in the Project

6.1 Port Polymorphism via Aegis-Morph

Aegis-Morph is a dynamic port reassignment mechanism that continuously rotates the listening ports of protected network services according to a defined schedule or trigger condition. By making service port assignments unpredictable and transient, this technique disrupts an attacker's ability to map the network and identify exploitable entry points through conventional probing strategies. The AI detection engine is designed with full awareness of the current port assignment state, ensuring

that legitimate traffic to dynamically assigned ports is not misclassified as suspicious.

6.2 Honeypot Integration

A honeypot is a deliberately exposed decoy system deployed within the network infrastructure. It mimics the appearance of a legitimate host or service while containing no actual operational data or functionality. Any interaction with the honeypot is, by definition, unauthorised and constitutes a high-confidence threat signal. Honeypot-captured events enrich the AI engine's threat intelligence, provide ground-truth labelled data for attacker behavioural analysis, and enable the generation of comprehensive attacker profiles including intent, methodology, and originating context.

7. Expected System Capabilities

Capability	Description
Real-Time Traffic Monitoring	Continuous capture and analysis of network flows with live dashboard indicators
AI-Driven Threat Detection	Machine learning model trained on labelled network traffic datasets to identify abnormal behaviours
Attacker Profiling	Automatic generation of a threat actor profile (network identity, behaviour pattern, threat level)
Graduated Alert System	Multi-level alerting mechanism triggered proportionally to confidence and severity of detected threats
Automated Blocking	Autonomous enforcement of firewall countermeasures upon confirmed threat detection, without human intervention
Port Polymorphism (Aegis-Morph)	Dynamic rotation of service port assignments to increase the cost and complexity of network reconnaissance
Honeypot-Based Deception	Decoy node that captures unauthorised interaction attempts and feeds behavioural data to the AI engine
Cross-Network Applicability	Architecture designed for generalisation across LAN, WAN, wireless, cloud, and hybrid environments

Table 1.5 — Expected System Capabilities

PART II

Detailed Needs Expression — Port Scanning, MTD & Honeypot

> **Document origin:** This part corresponds to the formal, finalised Needs Expression document (*PortScan_MTD_Honeypot_Final*), which constitutes the complete technical decision-making specification for the Aegis Entropy system.

1. Executive Summary

This document presents the formal decision-making needs expression for the design, implementation, testing, and validation of an Artificial Intelligence model dedicated to the real-time detection and automated mitigation of Port Scanning and Network Reconnaissance activity within networked environments.

Reconnaissance constitutes the first phase of virtually every cyberattack kill chain: an adversary probes the network to map active hosts, enumerate open ports, and fingerprint running services before launching a targeted intrusion. Early detection and containment at this phase can prevent the full materialisation of a subsequent, more destructive attack.

Beyond detection, this project integrates two advanced and complementary proactive defence mechanisms: **Moving Target Defence (MTD)**, which continuously mutates the network's observable attack surface to disrupt and mislead scanning activity, and **Honeypots**, which deploy decoy assets to attract, trap, and profile adversarial reconnaissance behaviour. Together, these three pillars — detection, deception, and surface mutation — form a layered, intelligent security posture.

2. Problem Statement

Port scanning, in its various forms — SYN scan, ACK scan, UDP scan, XMAS scan, FIN scan — generates traffic patterns that differ fundamentally from legitimate host-to-host communication. An adversary conducting a scan sends probes to hundreds or thousands of ports in succession, producing an anomalous distribution of half-open connections, RST responses, and unanswered packets.

Despite being a well-understood technique, detecting stealthy or slow-rate scanning — where the adversary deliberately spaces probes to evade threshold-based detection — remains a significant challenge. Such low-and-slow patterns require behavioural analysis over extended time windows rather than instantaneous packet-level inspection.

A further critical dimension is the static, predictable nature of conventional network configurations. Fixed IP addresses, stable port assignments, and consistent service layouts give adversaries a reliable map to exploit. Once a scan is complete, the attacker possesses a stable, actionable picture

of the infrastructure — a problem that no detection system alone can fully address.

3. Project Objectives

Objective	Description
Primary Objective	Train and deploy an AI model capable of detecting port scanning and network reconnaissance activity in real time across networked environments.
Scan Type Coverage	Detect and classify SYN Scan (stealth), TCP Connect Scan, UDP Scan, ACK Scan, XMAS Scan, and slow/distributed scanning variants.
Moving Target Defence	Integrate an MTD mechanism that continuously rotates network configuration parameters — including port assignments and IP addresses — to invalidate adversarial reconnaissance maps.
Honeypot Deception	Deploy honeypot nodes that mimic legitimate services to attract, trap, and observe adversarial scanning activity, enriching threat intelligence.
Attacker Profiling	Automatically build a structured profile for each detected threat actor: network identity, scan methodology, targeted scope, behavioural timeline.
Alerting	Trigger real-time, graduated alerts upon detection of scanning activity, classified by scan type, confidence score, and severity level.
Automated Response	Enforce automated protective measures — including network-level blocking — upon confirmed threat detection, without requiring human intervention.

Table 2.1 — Project Objectives

4. Functional Requirements

4.1 System Inputs

- Raw TCP/UDP packets captured on the network interface, filtered for partial or incomplete handshake patterns indicative of probing behaviour.
- Labelled training dataset containing port scanning and benign traffic samples (CICIDS2017 PortScan subset, UNSW-NB15 Reconnaissance category).
- Sliding time-window aggregated flow features computed per source IP over configurable observation windows.
- Network metadata: source/destination IP, MAC addresses, ports, protocol, TCP flag combinations, TTL values, and timestamps.
- Honeypot interaction logs: connection attempts, payloads, source identifiers, and session metadata from decoy nodes.
- MTD state feed: current port assignment table and IP rotation schedule, enabling the AI engine to correctly interpret traffic against the dynamic topology.

4.2 System Outputs

- Real-time dashboard: per-source port probe counts, scan heat map, active threat list, honeypot interaction events, MTD rotation log.

- Per-time-window classification: BENIGN or PORT_SCAN, with inferred scan type and associated confidence score.
- Attacker profile card: source IP and MAC, scan type, ports and hosts targeted, first/last seen timestamps, honeypot interaction flag, threat level.
- Graduated alert notification with scan type label, severity score, and contextual metadata.
- Automated network-level block rule applied to the confirmed attacker's source address, with the event logged to the incident database.
- MTD rotation trigger: upon detection, the system initiates a port/IP rotation cycle to further invalidate any reconnaissance data collected by the attacker.

5. Dataset Specification

Dataset	Relevant Subset	Approx. Samples	Source
CICIDS2017	PortScan category	~158,000 flows	UNB / CIC
UNSW-NB15	Reconnaissance category	~13,000 records	UNSW Australia
CAIDA Telescope	Backscatter / SYN flood probes	Variable captures	CAIDA.org

Table 2.2 — Training Dataset Specification

6. Key Network Features for the ML Model

Feature Name	Description	Relevance
Distinct Dst Ports/IP	# unique destination ports contacted by source	Critical — core scanning signature
SYN Flag Count	SYN packets sent without completing handshake	Critical — stealth SYN scan indicator
RST Flag Count	RST responses from closed ports	Very High — port probe rejection
Flow Duration	Duration of each individual probe connection	High — very short in fast scans
Total Fwd Packets	Packets per flow from the scanner	High — usually 1 per probe
ACK Flag Count	ACK packets sent without a prior SYN	Medium — ACK scan detection
IAT Mean	Mean inter-arrival time between probes	High — slow scans have large IAT
Unique Dst IPs/Src	# distinct destination hosts contacted	Medium — host discovery phase

TTL Value	Time-to-Live field in the IP header	Low — OS fingerprinting attempts
Bwd Packet Length	Response packet size	Medium — RST vs SYN-ACK ratio
Honeypot Interaction Flag	Binary: did source IP contact a decoy node	Critical — high-confidence threat signal
MTD Port Delta	Difference between probed port and current assigned port	High — reveals stale reconnaissance data

Table 2.3 — Key Network Features for the ML Model

7. Machine Learning Model Strategy

Component	Details
Primary Model	Random Forest Classifier — handles high-dimensional scan features with natural feature importance ranking and robustness to class imbalance.
Secondary Model	XGBoost — compared for accuracy on time-windowed aggregated features and as a high-performance ensemble benchmark.
Slow Scan Detection	Isolation Forest (unsupervised) applied on sliding 60-second window aggregates per source IP to capture low-and-slow probing patterns.
Feature Engineering	Time-window aggregation: distinct ports/IP/10s, SYN-ratio, IAT statistics, honeypot interaction count, and MTD port delta over rolling windows.
Imbalance Handling	SMOTE oversampling on the PORT_SCAN minority class in the training set.
Evaluation Metrics	Accuracy, Precision, Recall, F1-Score, AUC-ROC, Detection Rate per scan type, Honeypot True Positive Rate.
Validation Strategy	Stratified K-Fold (k=10) + temporal holdout (last 20% by time order).

Table 2.4 — Machine Learning Model Strategy

8. MTD & Honeypot Component Specifications

8.1 Moving Target Defence (MTD)

The MTD component continuously mutates key network configuration parameters to invalidate any reconnaissance data an adversary may have collected. The core principle is to impose a high and continuous cost on the attacker: a network map obtained through scanning becomes stale within minutes, forcing repeated re-scanning that is itself detectable.

Parameter	Details
Mutation Scope	Dynamic rotation of service port assignments and, optionally, virtual IP addresses of protected hosts within the network.

Rotation Trigger	Time-based (scheduled rotation interval) and event-based (triggered automatically upon confirmed scanning detection by the AI engine).
AI Integration	The MTD state table is fed as a live input to the AI model, ensuring that traffic directed at recently vacated ports is correctly identified as post-scan probing rather than legitimate communication.
Coordination	All legitimate nodes on the network are synchronised with the current MTD state via a secure distribution mechanism, ensuring zero disruption to authorised traffic during rotation cycles.

Table 2.5 — MTD Component Specifications

8.2 Honeypot Integration

Honeypot nodes are strategically deployed within the network topology as high-fidelity decoy systems. They present a realistic appearance to any scanning adversary while containing no operational data or functionality. All traffic directed to a honeypot is inherently unauthorised and constitutes an unambiguous threat signal.

Parameter	Details
Honeypot Type	Low-to-medium interaction honeypots emulating common services (SSH, HTTP, FTP) to attract and engage scanning and post-scan probing activity.
Placement Strategy	Distributed across the network topology to maximise the probability of contact during any broad or targeted scanning campaign.
AI Contribution	Honeypot interaction events — connection attempts, payloads, source IPs — are ingested as real-time features by the AI model, significantly boosting detection confidence for confirmed threat actors.
Profiling Output	Each honeypot interaction is logged and associated with the corresponding attacker profile, enriching the threat intelligence record with behavioural indicators and session-level metadata.

Table 2.6 — Honeypot Component Specifications

9. Non-Functional Requirements

Requirement	Criterion	Target Value
Fast Scan Detection	Detection time for aggressive Nmap scan	< 8 seconds
Slow Scan Detection	Detection time for stealthy / slow scan	< 90 seconds (sliding window)
Model F1-Score	On CICIDS2017 PortScan test subset	>= 88%
Model Accuracy	Overall classification on test set	>= 90%
False Positive Rate	Normal connections flagged as scans	< 6%
Dashboard Refresh	Scanner activity & honeypot heat map update frequency	<= 5 seconds

Blocking Response Time	From detection to automated block rule enforcement	< 15 seconds
MTD Rotation Latency	Time from detection trigger to completed port rotation	< 30 seconds
Honeypot Detection Rate	Proportion of scanning IPs interacting with at least one honeypot	>= 60%

Table 2.7 — Non-Functional Requirements & Performance Targets

10. Project Phases & Deliverables

Phase	Activities	Key Deliverables
1 — Conception	Network topology design, MTD architecture, feature set definition, scan type taxonomy.	Network diagram, MTD design spec, feature dictionary.
2 — Implementation	Dataset preprocessing, RF + XGBoost + Isolation Forest training, real-time capture pipeline, MTD rotation engine, honeypot deployment, dashboard.	Trained models (.pkl), IDS pipeline, MTD engine, honeypot nodes, dashboard UI.
3 — Testing	Simulate Nmap SYN, Connect, UDP, XMAS, slow scans; validate MTD disruption; test honeypot engagement.	Per-scan detection table, ROC curves, MTD effectiveness report, honeypot engagement log.
4 — Validation	K-Fold cross-validation, temporal holdout, end-to-end system validation including MTD and honeypot integration.	Final report, demo recording, multi-model comparison table.

Table 2.8 — Project Phases & Deliverables

11. Technical Stack Summary

Component	Technologies
OS / Virtualisation	Ubuntu Server (IDS + victim nodes), Kali Linux (attacker), GNS3 / VirtualBox
Programming Language	Python 3.10+
Traffic Capture	Scapy with BPF filters (SYN-only), PyShark, sliding window aggregation
ML Libraries	Scikit-learn, XGBoost, imbalanced-learn, Joblib, Pandas, NumPy
Web Dashboard	Flask + Socket.IO + D3.js (port heat map, honeypot event feed) / Grafana + InfluxDB
Blocking Engine	Linux iptables DROP rules via Python subprocess, with TTL-based auto-unblock scheduler
MTD Engine	Custom Python service managing dynamic port/IP rotation table, state distribution, and AI engine synchronisation

Honeypot Framework	Cowrie (SSH/Telnet), Dionaea (multi-service), or custom lightweight Python socket listeners
Attack Simulation	Nmap (SYN, ACK, UDP, XMAS, slow scans), Masscan, custom Scapy scripts
Version Control	Git + GitHub — feature branches per module component

Table 2.9 — Technical Stack Summary

12. Expected Outcomes & Success Criteria

✓ The trained model achieves an F1-Score >= 88% on the CICIDS2017 PortScan test subset.

✓ An aggressive Nmap SYN stealth scan is detected and an alert is triggered within 8 seconds.

✓ A slow-rate scan (1 probe/second) is correctly identified within the 90-second analysis window.

✓ The MTD engine completes a port rotation cycle within 30 seconds of a confirmed detection trigger.

✓ Following MTD rotation, continued probing of vacated ports by the same source IP is correctly classified as post-scan activity with no false negative.

✓ At least 60% of scanning source IPs interact with at least one honeypot node during a full scan campaign, generating enriched attacker profile data.

✓ The dashboard reflects scanning events, honeypot interactions, and MTD rotation status with a maximum update delay of 5 seconds.

✓ Automated blocking is enforced within 15 seconds of a confirmed detection.

✓ The attacker profile correctly identifies scan type (SYN / UDP / ACK / XMAS) and includes honeypot contact history where applicable.

✓ Legitimate traffic from authorised nodes generates a false positive rate below 6% across all test scenarios, including during active MTD rotation cycles.

PART III

Architecture & Conception — Aegis Entropy

> **Architecture Upgrade Note:** The original Needs Expression outlined a standard defensive posture. The final Aegis Entropy implementation elevates this to a **proactive, highly aggressive defence system**. This part documents every design upgrade, its technical rationale, and the three core pillars of the architecture.

1. Architecture Upgrades: Blueprint vs. Reality

The table below maps every original design choice from the Needs Expression document to its upgraded counterpart in the final Aegis Entropy system, along with the technical rationale for each upgrade.

Component	Original Specification	Aegis Entropy Upgrade
Response Engine	Linux iptables DROP rules via Python subprocess. Passive approach: the attacker's packets are silently discarded.	TCP Tarpit. Forged SYN-ACK with Window Size = 0 is returned, freezing tools and exhausting attacker resources.
MTD Mechanism	Dynamic rotation of service port assignments and virtual IP addresses.	Active Fingerprint Mutation. Traffic intercepted inline; TCP/IP headers rewritten to falsify OS identity.
Deception Layer	Full standalone honeypot VMs: Cowrie, Dionaea. High overhead.	Inline Shadow Nodes built with Python + Scapy. Fake responses generated directly from the network edge.

Table 3.1 — Architecture Upgrades: Original Specification vs. Aegis Entropy Implementation

> **Design Philosophy:** The shift from passive (DROP) to active (Tarpit + Mutation + Shadow Nodes) defence reflects a deliberate strategic choice: instead of simply blocking the attacker, Aegis Entropy wastes the attacker's time, corrupts their intelligence, and deceives them — all while the AI model continues to profile and classify the threat.

2. Pillar 1 — Core Infrastructure: Asynchronous Interception via NFQUEUE

The foundation of the Aegis Entropy active defence pipeline is the Linux kernel's Netfilter Queue (NFQUEUE) mechanism. This provides a user-space hook into the kernel's packet processing path, enabling Python scripts to intercept, inspect, and modify — or drop — individual network packets in real time.

Aspect	Description
Mechanism	When a packet reaches the NIC, the Linux kernel pauses its journey and hands it directly to the Python script via NFQUEUE.
Verdict Power	The packet cannot proceed until the script issues one of three verdicts: ACCEPT, DROP, or MODIFY.
Integration Point	All subsequent active defence mechanisms (Tarpit, Fingerprint Mutation, Shadow Node responses) are implemented as verdict decisions within this interception layer.
Performance	Asynchronous processing ensures that the main network stack is not blocked; only flagged packets are diverted to the Python handler.

Table 3.2 — NFQUEUE Interception Mechanism

3. Pillar 2 — Sabotage Engine: TCP Tarpitting

When the AI model flags an active Nmap scan, the Sabotage Engine engages: Scapy intercepts the scanning probe and crafts a forged **SYN-ACK response with Window Size = 0**. This is the core mechanism of TCP Tarpitting.

3.1 Technical Mechanism

A standard TCP connection requires both parties to negotiate a window size — the amount of data each end is willing to buffer. By returning Window Size = 0, the server signals that its buffer is full and the client must wait before sending data. The attacker's machine and scanning tools interpret this as: *"The server is alive but its buffer is full — I must wait."* Nmap will literally hang, frozen in a ESTABLISHED state, consuming one of its finite connection threads.

3.2 Operational Impact

- **Tool Freezing:** Nmap and similar scanners allocate a connection thread per probe. With Window Size = 0, the thread is blocked indefinitely, exhausting the scanner's parallelism.
- **Time Wasting:** A scan that would normally complete in seconds is stretched to minutes or hours.
- **Continued Profiling:** While the attacker is frozen, the AI model continues to accumulate behavioural data, refining the attacker profile and confidence score.
- **Deception:** The attacker receives a valid TCP response, confirming the port is open — but receives no actual service data.

4. Pillar 3 — Deception Engine: Active Fingerprint Mutation

The **network_mutator.py** script implements the Active Fingerprint Mutation mechanism. Rather than rotating ports (the original Aegis-Morph approach), this upgrade intercepts outgoing packets and dynamically rewrites TCP/IP header fields before they reach the attacker, creating conflicting and inconsistent OS signatures from a single virtual machine.

4.1 Mutation Scope & Parameters

Parameter	Description	Effect on Attacker
TTL Value	Dynamic rewriting of IP Time-to-Live field	Each probe response appears to originate from a different OS / hop count
TCP Options — MSS	Maximum Segment Size field in TCP options	Fingerprinting tools receive contradictory OS stack signatures
TCP Options — WScale	Window Scale factor in TCP handshake	Inconsistent with any single known OS profile
TCP Options — SACK	Selective Acknowledgement capability flag	Creates ambiguity in OS detection algorithms
TCP Options — Timestamps	Order and presence of TCP timestamp options	Disrupts timing-based OS fingerprinting (e.g., Nmap -O)

Table 3.3 — Active Fingerprint Mutation Parameters

4.2 Rotation Triggers

Fingerprint mutation is not static — it operates under two complementary trigger modes:

- **Time-Based Trigger:** Mutation parameters rotate on a scheduled interval, ensuring that even passive observers receive inconsistent fingerprint data over time.
- **Event-Based Trigger:** Upon a confirmed scan detection by the AI model, an immediate mutation cycle is initiated, invalidating any fingerprint data the attacker may have already collected.

4.3 AI Integration

The mutation state table is fed as a live input feature to the AI model. This ensures that traffic directed at fingerprinted ports is correctly classified in context — the model knows which OS profile is currently being projected and can interpret probe responses accordingly. This bidirectional integration between the deception layer and the detection engine is a key architectural innovation of Aegis Entropy.

> **Inline Shadow Nodes:** In addition to the three pillars above, Aegis Entropy replaces full honeypot VMs with lightweight **Inline Shadow Nodes** — Python socket listeners combined with Scapy packet crafting — that generate fake service responses (SSH, HTTP, FTP banners) directly from the network edge. This eliminates the overhead of full OS simulation while maintaining high-fidelity deception at the packet level.

PART IV

ML Data Dictionary

> **Document origin:** This part details the complete Machine Learning data dictionary for the Aegis Entropy system, including the final updated features introduced by the Aegis Entropy upgrades (Shadow Node Interaction and TCP Window Size outbound).

1. Network Features Data Dictionary

The following table details all network features extracted and used by the Machine Learning model (Random Forest / XGBoost / Isolation Forest) for detection and active response. Features marked **Updated** or **New** reflect the structural upgrades introduced by Aegis Entropy.

Feature Name	Type	Description & Relevance
Distinct Dst Ports/IP	Numeric	Number of unique destination ports contacted by the source. *Relevance: **Critical** — core scanning signature.*
SYN Flag Count	Numeric	SYN packets sent without completing the TCP handshake. *Relevance: **Critical** — stealth SYN scan indicator.*
RST Flag Count	Numeric	RST responses received from closed ports. *Relevance: **Very High** — port probe rejection indicator.*
Flow Duration	Numeric	Duration of each individual probe connection. *Relevance: **High** — usually very short in fast scans.*
Total Fwd Packets	Numeric	Total packets per flow originating from the scanner. *Relevance: **High** — usually 1 packet per probe.*
ACK Flag Count	Numeric	ACK packets sent without a prior SYN. *Relevance: **Medium** — ACK scan detection.*
IAT Mean	Numeric	Mean inter-arrival time between successive probes. *Relevance: **High** — slow scans exhibit large IAT values.*
Unique Dst IPs/Src	Numeric	Number of distinct destination hosts contacted by the source. *Relevance: **Medium** — indicates host discovery phase.*
TTL Value	Numeric	Time-to-Live field in the IP header. *Relevance: **Low** — used in OS fingerprinting attempts.*
Bwd Packet Length	Numeric	Response packet size (backwards direction). *Relevance: **Medium** — differentiates RST vs SYN-ACK ratio.*

Shadow Node Interaction *(Updated)*	Binary (Numeric)	Indicates whether the source IP contacted a decoy Inline Shadow Node. Replaces the original Honeypot Interaction Flag. *Relevance: **CRITICAL** — high-confidence threat signal.*
MTD Port Delta	Numeric	Difference between the probed port number and the currently assigned port in the MTD rotation table. *Relevance: **High** — reveals the use of stale reconnaissance data by the attacker.*
TCP Window Size (outbound) *(New)*	Numeric	Window size value in the SYN-ACK packet sent back to the attacker. *Relevance: **New** — metric introduced to evaluate TCP Tarpit effectiveness (Window Size = 0 signals active tarpitting).*

Table 4.1 — Complete ML Feature Data Dictionary (Aegis Entropy Final Version)

Data Dictionary Legend

Symbol / Highlight	Meaning
(Updated)	Feature definition or implementation updated in the Aegis Entropy upgrade from the original Needs Expression.
(New)	Feature introduced for the first time in the Aegis Entropy implementation, not present in the original specification.
Yellow row highlight	Updated feature (Shadow Node Interaction replaces Honeypot Interaction Flag).
Green row highlight	New feature (TCP Window Size outbound — Tarpit effectiveness metric).
CRITICAL relevance	Feature carries the highest predictive weight in the ensemble model; its presence or value is near-deterministic for threat classification.

Table 4.2 — Data Dictionary Legend

> **Model Usage Summary:** The 14 features above are consumed by the ensemble pipeline as follows: **Random Forest** and **XGBoost** operate on the full feature vector for per-flow binary classification. **Isolation Forest** operates on time-windowed aggregates (IAT Mean, Distinct Dst Ports/IP, SYN Flag Count, MTD Port Delta) for slow-scan anomaly detection. Shadow Node Interaction and MTD Port Delta serve as high-confidence override signals that can elevate a borderline BENIGN classification to PORT_SCAN regardless of other feature values.